Game Theory

Winning and losing states, Nim, Sprague-Grundy

Learning goals

- Classify positions as **winning (N)** or **losing (P)** with DP
- Spot **patterns/periods** by printing small tables
- Prove and use the **Nim** XOR rule (and its misère variant)
- Compute **Grundy numbers** with `mex` and combine games with XOR
- Recognize disguised Nim: staircase Nim, splitting games, games on DAGs
- Verify game claims with a **brute force**

The setting

Impartial, two-player, **perfect information**, no chance, finite:

- Both players have the **same moves** from every position
- **Normal play:** the player who cannot move **loses**
- Optimal play from both sides — "who wins?"

Not covered by today's theory: partisan games (chess: different pieces per player), games with chance, simultaneous moves.

Winning and losing positions

- **P-position** (Previous player wins): the player to move **loses**
- **N-position** (Next player wins): the player to move **wins**

Rules:

1. A terminal position (no moves) is **P**
2. A position is **N** iff **some** move leads to a P-position
3. A position is **P** iff **every** move leads to an N-position

Positions form a DAG (finite game) → evaluate in reverse topological order.

Example: subtraction game

> n sticks; a move removes 1, 3, or 4 sticks. Who wins for n = 0..10? (CSES 1729)

n	0	1	2	3	4	5	6	7	8	9	10
	L	W	L	W	W	W	W	L	W	L	W

```
vector<char> win(n + 1, 0);                  // win[0] = 0 (lose)
for (int i = 1; i <= n; i++)
    for (int p : moves)
        if (p <= i && !win[i - p]) { win[i] = 1; break; }
```

O(n · k). Pattern here: **period 7** (L at n ≡ 0, 2 mod 7).

Print the table!

Many game problems have n up to 10^{18}: the DP is impossible, but

1. Write the brute-force DP for small n
2. **Print** results, look for a period or a formula
3. Prove it (or at least verify it far beyond the samples)

Classic results found this way:

- Take 1..k from a pile: lose iff $n \equiv 0 \pmod{k+1}$
- Take 1, 3, 4: lose iff $n \bmod 7 \in \{0, 2\}$

Nim

n heaps $a_1, \ldots, a_n$. A move: pick one heap, remove **any positive** number of stones.

Bouton's theorem: the position is **losing** iff

$$a_1 \oplus a_2 \oplus \ldots \oplus a_n = 0$$

```
int x = 0;
for (int a : heaps) x ^= a;
puts(x ? "first" : "second");
```

(CSES 1730 Nim Game I)

Why XOR? (proof)

Let s = XOR of all heaps.

1. **s = 0 → every move gives s ≠ 0:** changing one heap $a_i \to a_i' \neq a_i$ changes s by $a_i \oplus a_i' \neq 0$
2. **s ≠ 0 → some move gives s = 0:** let b = highest set bit of s; pick a heap a_i with bit b set. Then $a_i \oplus s < a_i$ (bit b cleared, higher bits unchanged) → reduce a_i to $a_i \oplus s$.
3. Terminal (all zero) has s = 0

So {s = 0} is exactly the set of P-positions.

Nim: the winning move

Heaps 3, 4, 5: $s = 011 \oplus 100 \oplus 101 = \mathbf{010}$

- Highest bit of s: bit 1. Heap 3 = 011 has it.
- New size $3 \oplus 2 = 1$ → take 2 stones from heap 1

Check: $1 \oplus 4 \oplus 5 = 001 \oplus 100 \oplus 101 = 000$ ✓

Misère Nim

The player who takes the **last** stone **loses**.

- If **some heap ≥ 2**: same as normal Nim — first player wins iff XOR ≠ 0
- If **all heaps ≤ 1**: first player wins iff the number of 1-heaps is **even**

Intuition: play normal Nim until the moment you would leave only 1-heaps, then leave an odd number of them.

Sum of games

Several independent games played in parallel; a move = a move in **one** component. Who wins?

Knowing only W/L of each component is **not enough**:

- two W components (Nim heaps 1 and 1) → sum is L
- W + W (heaps 1 and 2) → sum is W

We need a finer invariant: the **Grundy number**.

Grundy numbers

mex(S) = smallest non-negative integer not in S. mex{0,1,3} = 2, mex{} = 0.

```
g(position) = mex { g(next) : next reachable in one move }
```

- g = 0 ⟺ P-position
- A position with Grundy value g **behaves exactly like a Nim heap of size g** (can move to any value < g; moves to larger values can be reversed)

Sprague-Grundy theorem

For a sum of independent impartial games $G_1 + \ldots + G_k$:

$$g(G_1 + G_2 + \ldots + G_k) = g(G_1) \oplus g(G_2) \oplus \ldots \oplus g(G_k)$$

First player wins iff the XOR is non-zero.

Recipe:

1. Split the game into independent components
2. Compute g for each component type (DP + mex)
3. XOR them

Grundy example: subtraction {1, 2, 3}

n	0	1	2	3	4	5	6	7	8
g	0	1	2	3	0	1	2	3	0

$g(n) = n \bmod 4$.

Several heaps, each move removes 1–3 from one heap (CSES 1098 Nim Game II): first player wins iff $\bigoplus (a_i \bmod 4) \neq 0$.

Computing Grundy values

```
vector<int> g(N + 1);
for (int x = 0; x <= N; x++) {
    vector<int> nxt = movesFrom(x);              // positions reachable from x
    vector<char> seen(nxt.size() + 1, 0);        // mex <= number of moves
    for (int y : nxt) if (g[y] <= (int)nxt.size()) seen[g[y]] = 1;
    int m = 0;
    while (seen[m]) m++;
    g[x] = m;
}
```

- g(x) ≤ number of moves from x — bounds the `seen` array
- **Splitting moves:** a heap → two heaps a, b gives the value g(a) ⊕ g(b)

Splitting games

> **Grundy's game** (CSES 2207): split a heap into **two unequal** non-empty heaps. Last mover wins.

```
g(n) = mex { g(a) ⊕ g(n − a) : 1 ≤ a < n − a }
```

$O(n^2)$ for all values up to n. For huge n: compute a table, inspect where g = 0 occurs; for this game no zero values appear after **n = 1222** (checked well beyond that by brute force).

Staircase Nim

> **CSES 1099 Stair Game:** stairs 1..n with p_k balls. Move: take ≥ 1 balls from stair k ≥ 2 to stair k − 1. Last mover wins.

Claim: equivalent to Nim on the **even-numbered** stairs.

- Moving from an even stair to an odd one = removing from a Nim heap
- Moving from an odd stair to an even one: the opponent can **move the same balls down again** → no effect on the outcome

```
int x = 0;
for (int k = 2; k <= n; k += 2) x ^= p[k];
puts(x ? "first" : "second");
```

Games on graphs

Token on a vertex of a directed graph; move along an edge; can't move → lose.

- **DAG:** W/L (or Grundy) in reverse topological order — Week 21
- **Graph with cycles:** some positions are **draws**. Retrograde BFS:
  - terminal vertices: L
  - a vertex with a successor L → W
  - a vertex whose **all** successors are W → L (count remaining successors)
  - whatever is never labeled → draw

Symmetry & strategy stealing

Short proofs that skip computation:

- **Mirror strategy:** in a symmetric position, copy the opponent's move (e.g., two equal Nim heaps; taking coins from a circle — the second player mirrors across the center)
- **Strategy stealing:** if an extra move never hurts, the first player cannot lose (Chomp, Hex)

Useful to **guess** an answer; still verify with brute force.

Complexity summary

Task	Cost
W/L DP, n states, k moves each	$O(n \cdot k)$
Nim	$O(n)$
Grundy values, n states, k moves	$O(n \cdot k)$
splitting game (all $a < n$)	$O(n^2)$
retrograde analysis with draws	$O(V + E)$

Common pitfalls

- Using W/L instead of Grundy for **sums** of games
- Forgetting that a terminal position has g = 0 (and is losing in normal play)
- Normal vs **misère** play mixed up
- `mex` array too small (g can be as large as the number of moves)
- Assuming a pattern after 10 values — check hundreds
- Game with cycles treated as a DAG (infinite loop or wrong answer)
- Not reading who moves first / who wins on the last move

Practice problems

- **CSES 1729** Stick Game · **AtCoder DP K** Stones
- **CSES 1730** Nim Game I · **CSES 1098** Nim Game II
- **CSES 1099** Stair Game · **CSES 2207** Grundy's Game
- **CSES 2208** Another Game (find the pattern!)
- **CF 768E** Game of Stones · **CF 603C** Lieges of Legendre
- **CF 455B** A Lot of Games (trie + game, from Week 29)
- **CF 1033G** Chip Game (hard, stretch)

Summary

- Terminal = losing; N iff some move to P; P iff all moves to N
- Print small tables → find periods/formulas → verify
- **Nim:** losing iff XOR = 0; winning move via the highest bit of the XOR
- **Grundy = mex** of successors; sums of games XOR their Grundy values
- Disguised Nim: staircase, splitting, subtraction games

Next week: square-root decomposition & Mo's algorithm.